A Strategic & Technical Reference for Enterprise AI Agents, Fixed-Seat Subscriptions, and Telemetry Integration

Audience: Executive & Technical Stakeholders | Scope: M365, Copilot Studio, Azure AI Foundry, Fabric | Status: Verified Technical Specification

Executive Overview

This document provides a research-backed reference on how AI agents and fixed-seat licenses operate across the Microsoft enterprise ecosystem. Every API endpoint, payload structure, and credential requirement documented below has been verified against official **Microsoft Learn** and **GitHub REST API** technical specifications.

It covers:
1. **The 4 Primary Agent Pathways in Microsoft**: Capabilities, exact raw API fields returned, administrator access, and authentication requirements.
2. **Enterprise AI Subscriptions (Fixed Seats)**: GitHub Copilot and Microsoft 365 Copilot pricing, raw API payloads vs. derived dashboard metrics (such as Acceptance Rate and Shelfware Detection).
3. **Practical Implementation Strategy With Current Resources**:
* **Fixed Seats Telemetry**: Ingesting real employee adoption and license efficiency data via Company Admin (Balaji).
* **Custom Autonomous Agents**: Code-first telemetry observability using Azure AI Foundry / Azure OpenAI.
4. **How to Identify Stakeholder GitHub Repositories**: Inspecting repository file trees to instantly categorize declarative, low-code, or code-first architectures.

1. The 4 Microsoft Agent Pathways (APIs, Exact Payloads & Admin Setup)

Pathway 1: Declarative / No-Code Agents (M365 Copilot & Agent Builder)

Configuration-driven assistants scoped to SharePoint, OneDrive, and Teams data defined via `declarativeAgent.json`.

- **How It Works**: Built using natural language or form fields inside Microsoft 365 Copilot Chat, Teams, or SharePoint. It wraps Microsoft 365 Copilot with custom instructions (up to 8,000 characters) and designated corporate data grounding.
- **Exact Exposed API / Endpoint**:
- **Microsoft Graph Usage Reports API**:
  `http GET https://graph.microsoft.com/v1.0/reports/getMicrosoft365CopilotUsageUserDetail(period='D30')`

`GET https://graph.microsoft.com/v1.0/reports/getMicrosoft365CopilotUserCountTrend(period='D30')`

- **Exact Raw Fields Returned by Microsoft Graph**:
- `reportRefreshDate`: Timestamp when the report data was refreshed by Microsoft.
- `userPrincipalName`: Authenticated employee email (e.g., `employee@company.com`).
- `displayName`: Full employee name.
- `lastActivityDate`: Date of most recent Copilot interaction across the enterprise.
- `copilotChatLastActivityDate`: Last active date in M365 Copilot Chat (Teams / Web).
- `microsoftTeamsCopilotLastActivityDate`: Last active date in Teams meetings / chats.
- `wordCopilotLastActivityDate`: Last active date inside Microsoft Word.
- `excelCopilotLastActivityDate`: Last active date inside Microsoft Excel.
- `powerPointCopilotLastActivityDate`: Last active date inside PowerPoint.
- `outlookCopilotLastActivityDate`: Last active date inside Outlook email drafting.
- `oneNoteCopilotLastActivityDate` & `loopCopilotLastActivityDate`: Activity in OneNote / Loop.
- *(Note: Microsoft strictly masks prompt text, conversation content, and raw token usage in M365 Copilot for enterprise compliance).*
- **Who Has Access**:
- Global Admin, Reports Reader, or Usage Summary Reports Reader in Microsoft 365 Admin Center (`admin.microsoft.com`).
- **Credentials & Permissions Needed**:
- Microsoft Entra ID (Azure AD) **App Registration**.
- `Tenant ID`, `Client ID`, `Client Secret`.
- Application Permission: `Reports.Read.All` (requires Admin Consent from Global Admin).
- **Per-User Granularity & Privacy Caveat**:
- **Yes, per employee**: Returns real emails (`userPrincipalName`) **UNLESS** the Microsoft 365 tenant has enabled "*Conceal user, group, and site names in all reports*" in Org Settings. If enabled, the admin must uncheck this setting to view actual employee emails instead of obfuscated GUIDs.

Pathway 2: Low-Code Workflow Agents (Microsoft Copilot Studio)

Autonomous chatbots and transactional business workflow agents built on Power Platform / Dataverse.

- **How It Works**: Built on Copilot Studio's visual workflow canvas. Interacts with enterprise software via 1,200+ Power Automate connectors, triggers automated actions, and manages conversational decision trees.
- **Exact Exposed API / Endpoint**:
- **Microsoft Dataverse OData Web API**:
`http GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/conversationtranscripts GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/bots`

- **Exact Raw Fields Returned by Dataverse (`conversationtranscript` entity)**:
- `conversationtranscriptid`: Unique GUID identifier for the conversation session.
- `name`: Transcript record title / session label.
- `conversationstarttime`: UTC timestamp when the user began the conversation.
- `bot_conversationtranscriptid`: Lookup linking the session to the specific Agent in the `bot` entity.
- `schematype` & `schemaversion`: Format version of the transcript JSON payload.
- `content`: Full conversation transcript stored in JSON format (contains user queries, bot responses, topic trigger tags, and session outcome tags: *Resolved*, *Escalated*, or *Abandoned*).
- `metadata`: Session metadata including client platform, participant roles, and batch indices.
- **Who Has Access**:
- Power Platform System Administrator, Environment Admin, or Bot Author in Copilot Studio Portal (`copilotstudio.microsoft.com`).
- **Credentials & Permissions Needed**:
- Entra App Registration added as an **Application User** in Power Platform Admin Center.
- Role: `System Customizer` or Read permissions on the `conversationtranscript` table.
- `Tenant ID`, `Client ID`, `Client Secret`, and `Dataverse Environment URL`.
- **Per-User Granularity**:
- **Authenticated Internal Sessions**: For bots deployed in Teams or SharePoint with SSO enabled, employee identity (`From.Name`, `From.Id`, user email) is captured in the `content` JSON. Public anonymous bots record anonymous session GUIDs.

Pathway 3: Code-First Custom Agents (Azure AI Foundry / OpenAI / App Insights)

Full-stack software agents developed by software engineers in Python or C# deployed to Azure cloud infrastructure.

- **How It Works**: Built using Semantic Kernel, AutoGen, or custom microservices connecting directly to Azure OpenAI deployments. The application code completely controls prompt orchestration, memory, and external database/API tool executions.
- **Exact Exposed APIs / Endpoints**:
- **Azure OpenAI Chat Completions API** (Direct Inference Telemetry):
  - Returned with every model inference execution:
    `json "usage": { "prompt_tokens": 128, "completion_tokens": 85, "total_tokens": 213 }`
- **Azure Application Insights REST API (Kusto / KQL Query)**:
  `http POST https://api.applicationinsights.io/v1/apps/{app-id}/query`
  - Queries standard tables:
  - `requests`: Incoming API calls (`duration`, `resultCode`, `success`, `timestamp`).
  - `dependencies`: Outgoing LLM and tool calls (`target`, `duration`, `resultCode`, `success`).

- `customDimensions`: Custom telemetry baggage injected by agent code (`user_email`, `task_category`, `minutes_saved`).

- **Azure Cost Management API**:
  `http POST https://management.azure.com/subscriptions/{subId}/providers/Microsoft.CostManagement/query?api-version=2023-11-01`
  - Returns: `PreTaxCost`, `UsageQuantity`, `MeterSubCategory` (e.g., `GPT-4o Standard`), `ResourceGroup`.
- **Who Has Access**:
- Azure Subscription Owner, Contributor, or Monitoring Reader on the Resource Group.
- **Credentials & Permissions Needed**:
- Azure Service Principal (Entra App Registration) with `Monitoring Reader` + `Cost Management Reader`.
- `Subscription ID`, `Resource Group`, `Tenant ID`, `Client ID`, `Client Secret`.
- **Per-User Granularity**:
- **100% Controllable in Code**: Azure OpenAI does not log end-user identity automatically. Our agent application code explicitly captures the authenticated employee email (`user_email`) from the session and logs it into OpenTelemetry / Application Insights baggage.

Pathway 4: Governed Data Agents (Microsoft Fabric Data Agents)

Specialized analytics agents built over Fabric OneLake, Lakehouses, and Power BI semantic models.

- **How It Works**: Converts natural language prompts into optimized DAX or SQL queries directly over enterprise data lakes.
- **Exact Exposed APIs / Endpoints**:
- **Power BI / Fabric Activity Events REST API**:
  `http GET https://api.powerbi.com/v1.0/myorg/admin/activityevents?startDateTime='...'&endDateTime='...'`
- **Fabric Capacity REST API**:
  `http GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}`
- **Exact Raw Fields Returned by Fabric Activity Stream**:
- `Activity`: Specific operation executed (e.g., `ExecuteDAXQuery`, `ExploreData`, `ViewReport`).
- `UserId`: Full User Principal Name (`employee@company.com`).
- `CapacityId` & `CapacityName`: Azure Fabric capacity assigned.
- `WorkspaceName` & `ArtifactName`: The specific Lakehouse, Warehouse, or Semantic Model queried.
- `ExecutionDuration`: Query processing duration in milliseconds.
- `CapacityUnitsConsumed (CU Seconds)`: Processing compute billed against Fabric capacity.
- **Who Has Access**:

- Fabric Capacity Administrator or Workspace Admin.
- **Credentials & Permissions Needed**:
- Entra App Registration with Fabric Admin permission `Tenant.Read.All`.
- **Per-User Granularity**:
- **Yes**: Full user principal names (`UserId: employee@company.com`) are logged in the activity audit stream.

---

2. Enterprise AI Subscriptions (Fixed Seats & Developer Tools)

Enterprises license developer and knowledge-worker AI tools primarily through **fixed seat monthly subscriptions**:

Subscription Tier	Enterprise Price	Telemetry Source API	Key Output Capabilities
GitHub Copilot Business	**$19 / user / mo**	GitHub REST API (`/copilot/billing/seats` & `/copilot/metrics/reports/ ...`)	Code completions in VS Code / JetBrains, seat assignment timestamps, IDE adoption, suggestion counts, acceptance counts.
GitHub Copilot Enterprise	**$39 / user / mo**	GitHub REST API (`/enterprises/{ent}/copilot/ ...`)	Everything in Business + PR summaries, organization-wide knowledge base indexing, custom code policies.
Microsoft 365 Copilot	**$30 / user / mo**	Microsoft Graph Reports API (`getMicrosoft365CopilotUsageUserDetail`)	Active vs. Inactive users, app breakdown (Teams, Word, Excel, Outlook), last active date per employee.
Microsoft Copilot Studio	**$200 / tenant / mo** *(25k msgs)*	Dataverse OData Web API (`conversationtranscripts`)	Total sessions, resolved vs escalated sessions, CSAT ratings, full message logs.

GitHub Copilot Telemetry: Raw API Payloads vs. Derived Metrics

GitHub Copilot does **not** return a pre-calculated "Acceptance Rate" field in its API response. Instead, GitHub provides **raw interaction counters** across two distinct REST APIs, and client dashboards calculate the derived efficiency metrics.

1. GitHub Copilot Seats API (License & Activity Status)

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/billing/seats`
- **Exact Raw Fields Returned**:
- `total_seats`: Total active seats provisioned for the organization.
- `seats[].assignee.login`: Developer GitHub username.
- `seats[].assignee.id`: Developer numeric ID.
- `seats[].created_at`: Timestamp when the seat license was assigned.
- `seats[].last_activity_at`: ISO timestamp of the developer's last active Copilot interaction.
- `seats[].last_activity_editor`: Exact editor and extension version (e.g., `vscode/1.94.0/copilot/1.240.0`).
- `seats[].plan_type`: License tier (`business` or `enterprise`).

2. GitHub Copilot Usage Metrics Reports API (Code Activity)

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/metrics/reports/users-28-day/latest`
- **Output Format**: Returns signed URLs to compressed **newline-delimited JSON (`.ndjson`)** report files.
- **Exact Raw Fields Inside the NDJSON File**:
- `day`: Activity date (`YYYY-MM-DD`).
- `user_login`: Developer username.
- `code_generation_activity_count` *(or `total_suggestions_count`)*: Total number of code completions suggested by Copilot.
- `code_acceptance_activity_count` *(or `total_acceptances_count`)*: Total number of code completions accepted by the developer.
- `loc_suggested_to_add_sum` *(or `lines_suggested`)*: Volume of code lines suggested by Copilot.
- `loc_added_sum` *(or `lines_accepted`)*: Volume of code lines accepted and retained by the developer.
- `totals_by_feature`: Breakdown by surface (`code_completion`, `chat_panel`, `pr_summary`).
- `totals_by_language_feature`: Breakdown by programming language (Python, TypeScript, C#, Go).

3. How Derived Dashboard Metrics Are Calculated

Dashboard Metric	Formula & Logic	Data Source	Purpose
Acceptance Rate (%)	`(code_acceptance_activity_count / code_generation_activity_count) * 100`	Calculated from Usage Metrics NDJSON	Measures developer trust and code relevance (industry benchmark is 25%–35%).
Lines Kept Rate (%)	`(loc_added_sum / loc_suggested_to_add_sum) * 100`	Calculated from Usage Metrics NDJSON	Measures the density of retained AI-generated code.
Shelfware / Inactive Waste ($)	If `(Current Date - last_activity_at) > 30 days`: `Wasted Spend = Inactive Seats * $19 (or $39)`	Calculated from Seats API (`last_activity_at`)	Identifies abandoned licenses for immediate reclaim or re-allocation.

3. Practical Implementation Strategy With Our Current Resources

> **Architecture & Ingestion Note**: Based on the raw output metrics received from each telemetry source, we design backend ingestion pipelines that transform, aggregate, and normalize these raw fields into high-level dashboard views (Scorecard, Cost Governance, and Seats & Adoption).

3.1 Fixed Seats Telemetry: What We Need From Balaji (Admin)

To display authentic employee adoption and identify inactive license cost savings, we only need from Balaji:
* **Option A: Microsoft 365 Copilot Usage CSV Export** from the M365 Admin Center (`admin.microsoft.com → Reports → Usage → Microsoft 365 Copilot → Export`), **OR**
* **Option B: Azure Entra App Registration credentials** (`Tenant ID` , `Client ID` , `Client Secret`) with `Reports.Read.All` read permission.

This provides the authentic list of licensed company employees, active usage dates, and inactive seats to quantify immediate cost-optimization savings.

3.2 Custom Autonomous Agents: Live Demonstration via Azure AI Foundry

We can build a simple, realistic enterprise agent that connects directly to an Azure AI Foundry / Azure OpenAI LLM resource to demonstrate live employee interactions and real usage metrics in real time:

1. **Live Employee Query Execution**:
2. The employee interacts with the agent to solve real workplace requests (e.g. IT device troubleshooting, access requests).
3. The query is processed live by the Azure AI Foundry LLM deployment.
4. **Real-Time Telemetry Extracted**:
5. **Employee Attribution**: Captured directly from the authenticated session (`user_email`).
6. **Exact Token Consumption**: Direct prompt tokens and completion tokens returned by Azure OpenAI.
7. **Actual Run Cost**: Real financial spend per execution based on Azure model deployment pricing.
8. **Handling Time Saved**: Standard manual task duration saved by auto-resolving the request (e.g. 25 minutes saved).
9. **Why This Works Best for Demos**:
10. It provides 100% verified, live results showing real employee usage without synthetic estimates.

---

4. How to Identify What Type of Agents Are in Your Stakeholder's GitHub Repo

When your stakeholder shares a GitHub repository containing AI agents, inspect its file tree to immediately determine its architecture:

```
Stakeholder GitHub Repository
│
├── CASE A: Declarative Agent (M365 Copilot Extension)
│   ├── appPackage/
│   │   ├── manifest.json
│   │   └── declarativeAgent.json   ⟵ Look for this manifest file!
│   └── (No executable backend code; configuration package only)
│
├── CASE B: Copilot Studio Solution (Power Platform Unpack)
│   ├── Solution.xml
│   ├── bots/                       ⟵ Look for bots & botcomponents folders!
│   └── botcomponents/
│   └── (Exported low-code solution; runs in Dataverse)
│
└── CASE C: Code-First Custom Agent (Foundry / Semantic Kernel)
    ├── requirements.txt / pyproject.toml
    ├── semantic_kernel/ or azure.ai
    ├── plugins/
    └── main.py / agent.py          ⟵ Full Python/C# execution logic!
```

Action Steps Based on Repo Structure:

1. **If Case A (Declarative Agent)**:
2. Do not run the repo code. It is an M365 manifest.
3. Ask the admin for an Entra App Registration to pull usage data from the **Microsoft Graph Reports API**.
4. **If Case B (Copilot Studio Export)**:
5. It is a Power Platform solution.
6. Connect to their **Dataverse Web API** (`conversationtranscripts`) to ingest session outcomes.
7. **If Case C (Code-First Custom Agent)**:
8. It is custom software running on Azure.
9. Connect to their **Azure Application Insights** or configure a lightweight webhook in their agent loop.
